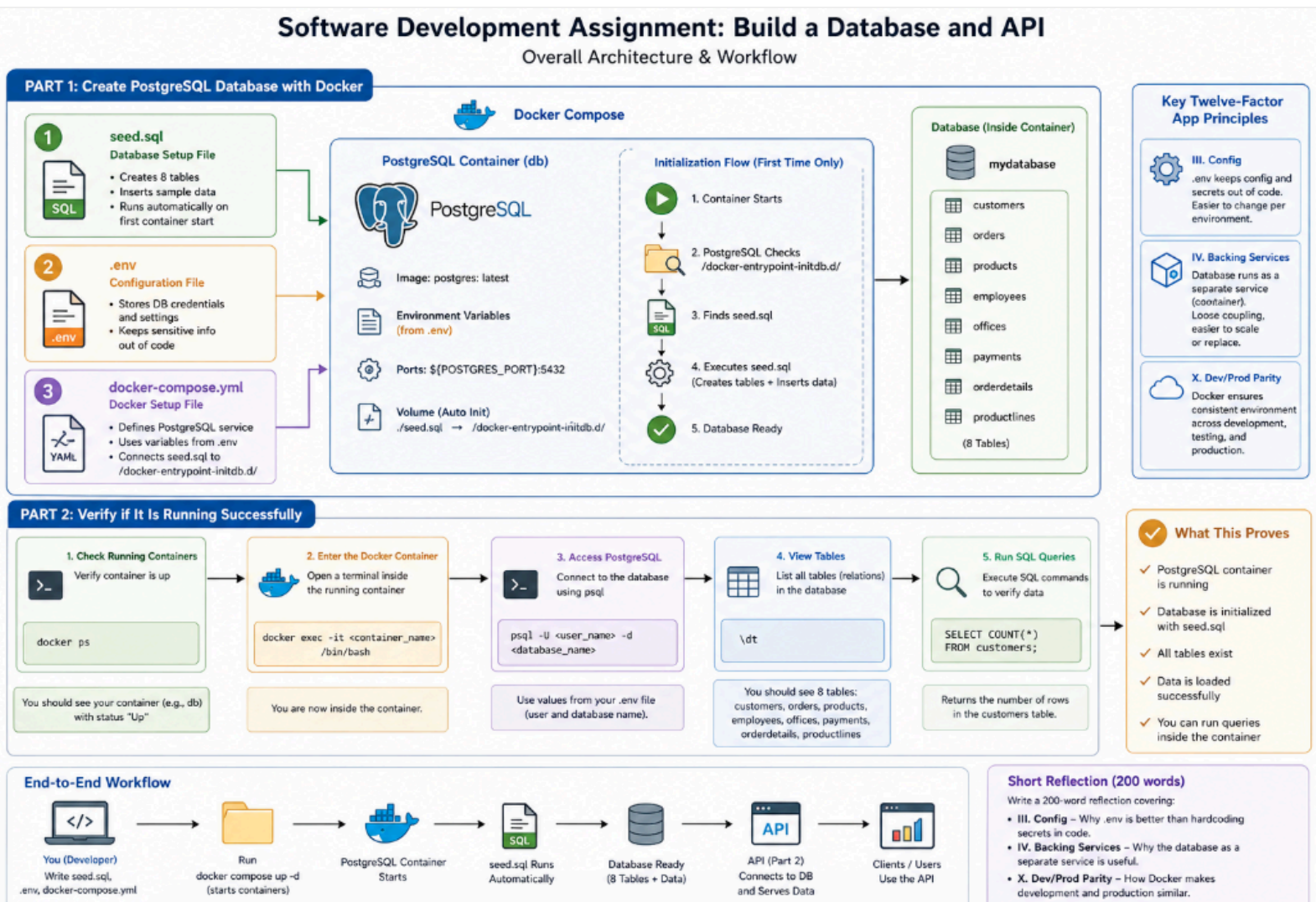


Software Development Assignment: Build a Database and API (Simple Guide)



This assignment has two main parts:

1. Create a database using Docker
2. Build an API to access the data

Part 1: Create a PostgreSQL Database with Docker

Goal

You will run a PostgreSQL database using Docker and automatically load data when it starts.

Step 1: Use seed.sql file which is provided.

1. seed.sql (Database Setup File)

- This file contains SQL statements used to initialize the database.
- It creates the required tables, including:
 - customers
 - orders
 - products
 - offices
 - employees
 - productlines
 - payments
 - orderdetails
- It inserts sample data into all the above tables.
- When the project is run using Docker Compose command, the `seed.sql` file should execute automatically.
- This ensures that the database is created, all tables are set up, and initial sample data is inserted without any manual steps.

Table	Rows
productlines	7
products	110
offices	7
employees	23
customers	122
orders	326
payments	273
orderdetails	2,996
Total	3,864

2. **.env** (Set up configuration file)

- This file stores sensitive information like database username, password, hostname, port and so forth.
- Example:

```
POSTGRES_USER=admin
POSTGRES_PASSWORD=secret
POSTGRES_DB=mydatabase
POSTGRES_PORT=5432
```

- Do NOT write these values directly in your code.
- The provided example has placeholder values, do not literally use these values while setting up a postgres connection.

3. **docker-compose.yml** (Setup File for Docker)

- This file tells Docker how to run PostgreSQL.
- Use variables from **.env** like this:

Follow these simple instructions:

- Create a service for your database (name it something like **db**).
- Use the official PostgreSQL image.

- Do **not** write username, password, or database name directly → use variables from `.env` like:
`${POSTGRES_USER}`

Important Setup Steps

- **Connect your SQL file**
 - You must link your `seed.sql` file to this folder inside the container:
`/docker-entrypoint-initdb.d/`
 - This is the special location where PostgreSQL looks for setup scripts.
-

What This Does

- When you run your container for the first time:
 - PostgreSQL starts
 - It checks that folder
 - It finds `seed.sql`
 - It runs it automatically
 - Your tables and data are created
-

In simple words:

You are telling Docker:

“Start PostgreSQL, use my settings from `.env`, and run my SQL file automatically when the database is created.”

Step 2: Short Reflection (200 words)

Write a simple explanation:

Config (Factor III)

- Why is `.env` better than writing passwords in code?

Backing Services (Factor IV)

- Why is treating the database as a separate service useful?

Dev/Prod Parity (Factor X)

- Why does Docker make development and production similar?

Part 2: Verify if it is running successfully

Table	Rows
productlines	7
products	110
offices	7
employees	23
customers	122
orders	326
payments	273
orderdetails	2,996
Total	3,864

Once PostgreSQL starts successfully, your database should contain 8 tables, including the **customers** table.

After running Docker, you can verify that your container is up and running by using the appropriate command to check its status.

```
(base) fm-pc-lt-148@Acer-Predator:~/Desktop/AI_Fellowship/week2_tasks/fastapi_app$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS
ORTS          NAMES
77ef99dec794   fastapi_app-api                    "uvicorn main:app --..." About an hour ago Up About an hour
.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp   fastapi_app-api-1
fa53fc83a46b   postgres:latest                    "docker-entrypoint.s..." About an hour ago Up About an hour (healthy)
.0.0.0:5433->5432/tcp, [::]:5433->5432/tcp   fastapi_app-db-1
```

You can verify whether the container is running and confirm that PostgreSQL is operating in a separate container. Then, you can access that container to check if the tables have been properly populated using the available commands.

```

fa53fc83a46b postgres:latest "docker-entrypoint.s..." About an hour ago Up About an hour (healthy) 0.0.0.0:5433->5432/tcp,
[::]:5433->5432/tcp fastapi_app-db-1
(base) fm-pc-lt-148@Acer-Predator:~/Desktop/AI_Fellowship/week2_tasks/fastapi_app$ docker exec -it fa53fc83a46b /bin/bash
root@fa53fc83a46b:/# psql -U postgres -d classicmodels
psql (18.3 (Debian 18.3-1.pgdg13+1))
Type "help" for help.

classicmodels=# \dt
               List of tables
Schema |      Name      | Type  | Owner
-----+-----+-----+-----
public | customers      | table | postgres
public | employees      | table | postgres
public | offices        | table | postgres
public | orderdetails   | table | postgres
public | orders         | table | postgres
public | payments       | table | postgres
public | productlines   | table | postgres
public | products       | table | postgres
(8 rows)

classicmodels=# select count(*) from customers;
 count
-----
    122
(1 row)

```

1. Enter the Docker Container

Use this command to open a terminal inside your running container:

```
<code>docker exec -it <container_name> /bin/bash</code>
```

2. Access PostgreSQL

Once inside the container, connect to the database using:

```

<code>
    psql -U <user_name> -d <database_name>
</code>

```

- Use the same **user_name** and **database_name** that you defined in your **.env** file.
-

3. View Tables in the Database

To see all tables (relations), run:

```
<code>\dt</code>
```

4. Run SQL Queries

You can now execute SQL commands.

Example: Count number of rows in the **customers** table:

```
<code>SELECT COUNT(*) FROM customers;</code>
```

Simple Summary

1. Enter container
2. Connect to PostgreSQL
3. List tables
4. Run queries